MORC, the Mora Recursive Calculator

A small lazy language and its translation to combinator graphs

October 27, 2026

Abstract

MORC is a small functional language with integers, functions and lists, in the tradition of Turner's SASL and Miranda. It has named definitions, infix operators with the usual precedences, **if**–**then**–**else**, anonymous functions, and lazy evaluation. It has no run-time system of its own: a MORC program is translated into a graph of Turner's combinators, which a graph reducer of under two and a half kilobytes, written for the VM16 virtual machine, rewrites into a result. This paper defines the language and follows a program through every step of the translation: the tokens, the grammar and its precedences, the reduction of the syntax to a core λ-calculus, bracket abstraction, the linking of definitions into one cyclic graph, the text format in which the graph is handed to the reducer, and the reduction itself. It ends with measurements and with what the language still lacks.

Contents

1 Introduction 2

2 The language by example 3

3 Lexical structure 3

4 Grammar 4

5 Meaning 5

6 From MORC to the core 6

7 Bracket abstraction 7

8 Linking definitions 8

9 The graph file 9

10 Reduction 10

11 Measurements 11

12 What MORC lacks 11

13 Summary 12

A Reference card 13

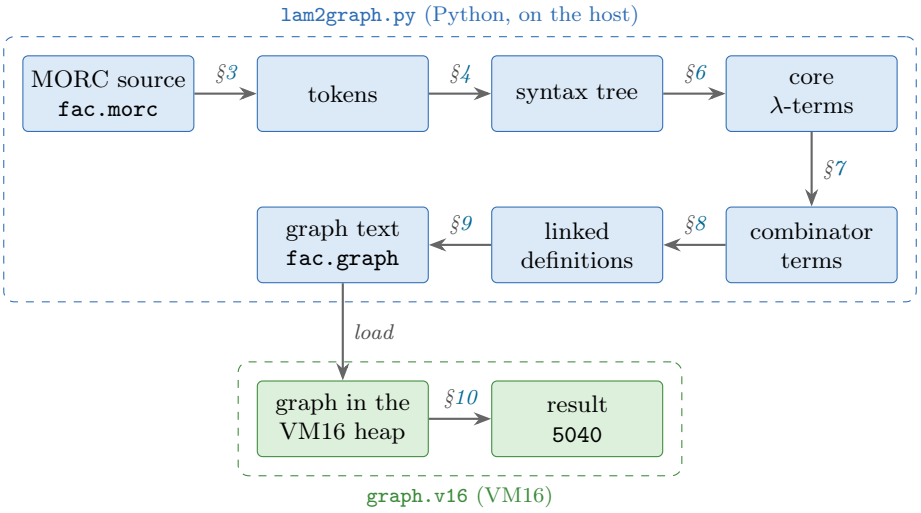


Figure 1: The path of a MORC program. Everything up to the graph text happens on the host, in `lam2graph.py` and `morc.py`; the reducer on VM16 only loads a graph and rewrites it. `run` does both steps.

1 Introduction

A lazy functional language can be implemented in a surprisingly small amount of code if one is willing to go through combinators. Turner's SASL [4] and later Miranda [5] compiled a program into a graph built from a handful of fixed functions, S, K, I, B, C and a few more, and evaluated it by rewriting that graph in place. No environments, closures or stack frames are needed; the graph is the whole state of the computation.

MORC applies that idea to a machine with 64 kilobytes of memory. The compiler runs on a host computer, in Python, and the reducer runs on VM16, a 16-bit virtual machine designed to be implemented on 8-bit processors such as the Z80. Figure 1 shows the path of a program. This is MORC's factorial:

```
fn fac n = if n = 0 then 1 else n * fac (n - 1);
fn main = fac 7;
```

```
$ ./run fac.morc
5040
[87 reductions, 0 collections]
```

The rest of the paper explains what happens between those two listings. Section 2 introduces the language through examples, sections 3 and 4 define its tokens and grammar, and section 5 its meaning. Sections 6 to 8 give the translation: from MORC to a core λ-calculus, from there to combinators, and from a set of definitions to one graph. Sections 9 and 10 show how the graph reaches the reducer and how it is reduced, and sections 11 and 12 measure the result and discuss what is missing. A companion tutorial [1] explains combinator graph reduction itself in more depth.

Where MORC comes from. The name stands for *Mora Recursive Calculator*. MORC began as a grammar sketched some years before this implementation, with **fn** definitions, **if**–**then**–**else** and infix operators. The version described here keeps that surface and settles the details the sketch left open, chiefly the precedence of function application, in the way most functional languages do. The same compiler also reads a terser notation, the `.lam` files of the companion tutorials, in which MORC's factorial is written `fac n = if n == 0 then 1 else n * fac (n-1); fac 7`.

Both produce the same syntax tree, and everything after the reader is shared.

2 The language by example

A MORC program is a series of definitions, each introduced by **fn** and ended by a semicolon. The value of the program is the value of *main*.

Functions on numbers. Numbers are integers, and functions are defined by naming their parameters:

```
fn fib n = if n < 2 then n else fib (n - 1) + fib (n - 2);
fn main = fib 20;
```

Function application is written by juxtaposition and binds tighter than any operator, so the **else** branch adds the results of two calls. A definition may use itself, and definitions may use each other in any order.

Functions as values. A function can be passed, returned and written anonymously with a backslash and an arrow:

```
fn twice f x = f (f x);
fn main = twice (\x -> x * 2) 3;          # 12
```

Every function takes its arguments one at a time, so `twice (\x -> x * 2)` is itself a function, waiting for its second argument. An operator in brackets is a function of two arguments: `(+) 3 4` is 7.

Lists. `h : t` is a list with head h and tail t, and **nil** is the empty list. A list is taken apart by applying it to two things: a function of the head and the tail, used if the list is not empty, and a value to give if it is:

```
fn length l = l (\h t -> 1 + length t) 0;
fn main = length (1 : 2 : 3 : nil);       # 3
```

Laziness. An argument is evaluated only when its value is needed, and then only once. Lists can therefore be infinite, as long as only a finite part of them is used. This program finds the 100th prime by filtering the infinite list of integers from 2:

```
fn from n = n : from (n + 1);
fn filter p l = l (\h t -> if p h then h : filter p t else filter p t) nil;
fn sieve l = l (\h t -> h : sieve (filter (\x -> x % h != 0) t)) nil;
fn nth k l = l (\h t -> if k = 0 then h else nth (k - 1) t) 0;
fn primes = sieve (from 2);
fn main = nth 99 primes;                  # 541
```

primes is a definition without parameters whose value is an infinite list. It is computed only as far as *nth* looks, and what has been computed is kept and shared.

3 Lexical structure

A program is read as a sequence of tokens, separated by any amount of white space. A `#` starts a comment that runs to the end of the line. The tokens are:

```
program    ::= definition { definition }
definition ::= fn NAME { NAME } = expr ;

expr       ::= \ NAME { NAME } -> expr
             | if expr then expr else expr
             | compare
compare    ::= cons [ (= | == | != | < | <= | > | >=) cons ]
cons       ::= sum [ : cons ]
sum        ::= [ - ] product { (+ | -) product }
product    ::= app { (* | / | %) app }
app        ::= atom { atom }
atom       ::= NAME | NUMBER | true | false | nil
             | ( expr ) | ( operator )
             | \ ... | if ...                    (as in expr)
```

{ x } is zero or more x, and [x] is an optional x. The semicolon after the last definition may be left out.

Figure 2: The grammar of MORC.

level	operators	grouping	example
loosest	`\x -> e`, **if**	extend to the right	`\x -> x + 1` is `\x -> (x + 1)`
	`= == != < <= > >=`	none	`a < b < c` is an error
	`:`	to the right	`1 : 2 : nil` is `1 : (2 : nil)`
	`+ -`	to the left	`10 - 3 - 2` is 5
	`* / %`	to the left	`1 + 2 * 3` is 7
tightest	application	to the left	`f x y` is `(f x) y`; `f x + 1` is `(f x) + 1`

Table 1: Precedence and grouping, loosest first. These are the conventions of Haskell [2] and most other functional languages.

token	form	examples
name	a letter, then letters, digits and `_`	`fac`, `x1`, `is_even`
number	one or more digits	`0`, `16383`
keyword	**fn if then else true false nil**	
operator	`+ - * / % : = == != < <= > >=`	
punctuation	`\ -> ( ) ;`	

Tokens are found by taking the longest match at each point, so `<=` is one token and `n-1` is three. Keywords are names that cannot be used for anything else. Numbers have 15 bits, the range the reducer stores (section 10); a literal outside −16384 to 16383 wraps around, as arithmetic does. The tokens of the factorial program are

```
fn fac n = if n = 0 then 1 else n * fac ( n - 1 ) ; fn main = fac 7 ;
```

4 Grammar

Figure 2 gives the grammar. Each level of *expr* handles one group of operators, from the loosest at the top to the tightest at the bottom, which is the usual way of building precedence into a grammar for a recursive-descent parser.

Table 1 summarises the result. Several of the choices deserve a comment.

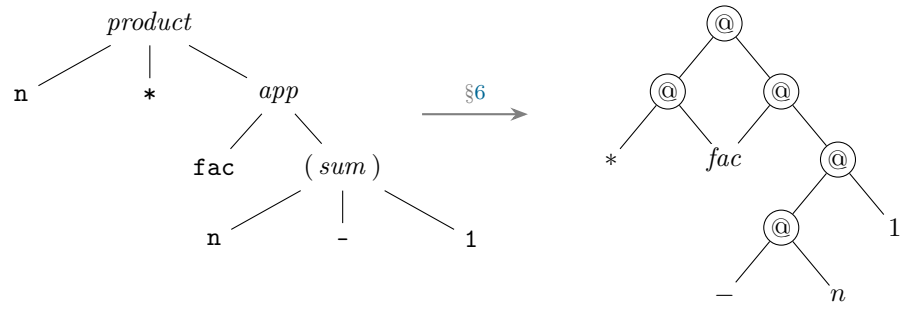


Figure 3: `n * fac (n - 1)`: its parse tree (left), and the core term it becomes (right), $*\ n\ (fac\ (-\ n\ 1))$, where every @ is an application of its left child to its right.

Application binds tightest. `fac (n - 1)` is a call, and `n * fac (n - 1)` multiplies n by it. The brackets are needed because `fac n - 1` would mean `(fac n) - 1`. In *app*, a sequence of atoms is one application, grouped to the left, so `f x y` applies `f x` to y.

Lambdas and conditionals extend to the right. The body of `\x -> ...` and the **else** branch of an **if** are whole expressions, so they reach as far right as they can: `\x -> x + 1` is a function that adds one, and `if c then a else b + 1` adds one to b only. Because a lambda or an **if** may also appear where an atom is expected, they need no brackets as the last argument of a call or the right operand of an operator: `map \x -> x * 2` and `1 + if c then 2 else 3` both parse as expected. Nested conditionals chain without brackets:

```
fn ack x y = if x == 0 then y + 1
             else if y == 0 then ack (x - 1) 1
             else ack (x - 1) (ack x (y - 1));
```

Comparisons do not chain. `a < b < c` is rejected rather than read as `(a < b) < c`, which would compare a truth value with c. MORC accepts both `=` and `==` for equality. There is no ambiguity with the `=` of a definition, which can only follow **fn** and the names of the parameters.

Lists group to the right. `:` sits between the comparisons and the arithmetic, so `n + 1 : rest` puts $n + 1$ in front of *rest*, and `1 : 2 : nil` is a list of two elements.

Minus. A `-` at the start of a *sum* is negation: `-x` is $0 - x$, and `-5` is the number -5. Elsewhere `-` is subtraction, so `n-1` and `n - 1` mean the same. To pass a negative number as an argument it needs brackets, `f (-1)`, since `f -1` is f minus one.

Operators as functions. An operator alone in brackets, `(+)`, `(<=)` or `(:)`, is the function of two arguments that applies it.

The grammar can be read by a recursive-descent parser with one token of lookahead, one procedure per level, and that is how `morc.py` reads it. Figure 3 shows the tree for the **else** branch of the factorial.

5 Meaning

Values. There are two kinds of value: integers and functions. Integers have 15 bits, from -16384 to 16383, and arithmetic wraps around. `/` and `%` truncate towards zero, so `-7 / 2` is -3 and `-7 % 2` is -1; dividing by zero stops the program.

Truth values are numbers. The comparisons return 1 for true and 0 for false, **true** and **false** are 1 and 0, and **if** takes its **then** branch for any number other than 0. A truth value can therefore be used in arithmetic: `count + (x > 0)`.

Lists are functions. `h : t` is the function that, given c and n, returns $c\ h\ t$, and **nil** is the function that returns n. This is the *Scott encoding* of lists. Taking a list apart is applying it, as in *length* above, and a list is printed as a function if it is the result of the program.

Laziness. An expression is evaluated only when its value is needed: by an operator, which needs numbers; by **if**, which needs its condition; or by an application, which needs a function. Once evaluated, its value replaces it, so it is never evaluated twice. In particular, the branches of an **if** need no special treatment: only the chosen one is ever evaluated, because only it is needed.

Definitions are evaluated at most once too. A definition with parameters is a function; one without, such as *primes*, is a value that is computed the first time it is used and then kept for the rest of the run.

Errors. The compiler reports a syntax error with the token where it occurred, an unbound name, a name defined twice, and a missing *main*. At run time the reducer stops with a message for division by zero, an operator or **if** given something other than a number, a number applied as if it were a function, a value defined as itself (`fn x = x`), and running out of stack or memory.

6 From MORC to the core

The first step of the translation removes everything that is particular to MORC. What remains is a small *core* language, the untyped λ-calculus with numbers and constants:

$$e \ ::= \ x \mid n \mid c \mid e\ e \mid \lambda x.\, e \qquad c \in \{+,\ -,\ *,\ /,\ \%,\ =,\ <,\ \mathsf{IF}\}.$$

The constants are curried: $+\ a\ b$ adds a and b, $<\ a\ b$ is 1 if $a < b$ and 0 otherwise, and $\mathsf{IF}\ c\ t\ e$ is t or e. This is the language that the bracket abstraction of section 7 works on, and the language of the `.lam` files.

Table 2 gives the translation $[\![\cdot]\!]$. It works on the tree that the parser builds and is applied to each definition in turn. Most of it is direct: an infix operator becomes the corresponding constant applied to the two operands, and **if** becomes the constant IF.

Four comparisons from two. The reducer has only $=$ and $<$. $a > b$ is $b < a$, with the operands swapped, and the other three negate a comparison: $1 - (a < b)$ is $a \geq b$, since the comparison is 0 or 1. The negation costs a subtraction at run time, which section 11 measures. Swapping the operands of $>$ changes which operand is evaluated first. In a language without side effects this cannot change a result, only which of two errors is reported if both operands fail.

Lists from two definitions. *cons* and *nil* are ordinary definitions that the compiler adds to every program. Their names, "`:`" and "`nil`", cannot be written as MORC names, so they cannot clash with the program's own, and like every other definition they are only included in the output if the program uses them (section 8).

No new constructs. Every MORC construct becomes a core term built from things the reducer already has, so the reducer did not change when MORC was added. This is also why MORC and the `.lam` notation can share everything after the parser.

MORC	core
fn $f\ x_1 \ldots x_n$ `=` e	$f = \lambda x_1 . \ldots \lambda x_n . [\![e]\!]$
`\`$x_1 \ldots x_n$ `->` e	$\lambda x_1 . \ldots \lambda x_n . [\![e]\!]$
$e_1\ e_2$	$[\![e_1]\!]\ [\![e_2]\!]$
if c **then** a **else** b	$\mathsf{IF}\ [\![c]\!]\ [\![a]\!]\ [\![b]\!]$
$a\ op\ b$, for op in `+ - * / % = <`	$op\ [\![a]\!]\ [\![b]\!]$
a `==` b	$=\ [\![a]\!]\ [\![b]\!]$
a `>` b	$<\ [\![b]\!]\ [\![a]\!]$
a `>=` b	$-\ 1\ (<\ [\![a]\!]\ [\![b]\!])$
a `<=` b	$-\ 1\ (<\ [\![b]\!]\ [\![a]\!])$
a `!=` b	$-\ 1\ (=\ [\![a]\!]\ [\![b]\!])$
`-`a	$-\ 0\ [\![a]\!]$, or the number $-n$ if a is a number n
true, **false**	1, 0
a `:` b	$cons\ [\![a]\!]\ [\![b]\!]$
nil	nil
`(`op`)`	op, or $\lambda a\, b . [\![a\ op\ b]\!]$ if op is not a constant
built in	$cons = \lambda h\, t\, c\, n . c\ h\ t \qquad nil = \lambda c\, n . n$

Table 2: The translation from MORC to the core language.

7 Bracket abstraction

The core language still has variables and λs, and the reducer has neither. Bracket abstraction removes them. For a variable x and a term e without λs, $[x]\, e$ is a term without x that, applied to x, behaves like e:

$$[x]\, x = \mathsf{I} \qquad [x]\, e = \mathsf{K}\ e \ \ (e \text{ any other variable or constant}) \qquad [x]\, (e_1\ e_2) = \mathsf{S}\ ([x]\, e_1)\ ([x]\, e_2),$$

where $\mathsf{I}\, x = x$, $\mathsf{K}\, x\, y = x$ and $\mathsf{S}\, f\, g\, x = f\, x\, (g\, x)$. A term $\lambda x.M$ is compiled by first compiling M, which removes the λs inside it, and then abstracting x.

These three rules alone produce large terms, so each S is simplified as it is built, by Turner's rules [4], tried in this order:

$$\begin{array}{ll} \mathsf{S}\,(\mathsf{K}\,p)\,(\mathsf{K}\,q) = \mathsf{K}\,(p\,q) & \mathsf{S}\,(\mathsf{B}\,p\,q)\,(\mathsf{K}\,r) = \mathsf{C}'\,p\,q\,r \\ \mathsf{S}\,(\mathsf{K}\,p)\,\mathsf{I} = p & \mathsf{S}\,p\,(\mathsf{K}\,q) = \mathsf{C}\,p\,q \\ \mathsf{S}\,(\mathsf{K}\,p)\,(\mathsf{B}\,q\,r) = \mathsf{B}^*\,p\,q\,r & \mathsf{S}\,(\mathsf{B}\,p\,q)\,r = \mathsf{S}'\,p\,q\,r \\ \mathsf{S}\,(\mathsf{K}\,p)\,q = \mathsf{B}\,p\,q & \end{array}$$

The new combinators are defined by

$$\begin{array}{ll} \mathsf{B}\,f\,g\,x = f\,(g\,x) & \mathsf{S}'\,k\,f\,g\,x = k\,(f\,x)\,(g\,x) \\ \mathsf{C}\,f\,g\,x = f\,x\,g & \mathsf{B}^*\,k\,f\,g\,x = k\,(f\,(g\,x)) \\ & \mathsf{C}'\,k\,f\,g\,x = k\,(f\,x)\,g \end{array}$$

and each one handles a common case directly: B when only the function part of an application uses x, C when only the argument does, and the primed versions for the same cases one level deeper. The companion tutorial [1] derives them and proves the rules correct.

Here is what some MORC fragments become, as `lam2graph.py -t` prints them:

MORC	combinators
`\x y -> y x`	$\mathsf{C}\ \mathsf{I}$
`fn twice f x = f (f x)`	$\mathsf{S}\ \mathsf{B}\ \mathsf{I}$
`\x -> x * 2`	$\mathsf{C} * 2$
`fn f a b = a > b`	$\mathsf{C} <$
`fn f a b = a <= b`	$\mathsf{B}\ (\mathsf{B}\ (-1))\ (\mathsf{C} <)$
`fn f x = -x`	-0
`fn len l = l (\h t -> 1 + len t) 0`	$\mathsf{C}\ (\mathsf{C}\ \mathsf{I}\ (\mathsf{K}\ (\mathsf{B}\ (+1)\ \mathit{len})))\ 0$
cons, that is `(:)`	$\mathsf{B}^*\ (\mathsf{B}\ (\mathsf{B}\ \mathsf{K}))\ \mathsf{C}\ (\mathsf{C}\ \mathsf{I})$
nil	$\mathsf{K}\ \mathsf{I}$
`fn fac n = if n = 0 then 1 else n * fac (n - 1)`	$\mathsf{S}\ (\mathsf{C}'\ \mathsf{IF}\ (\mathsf{C} = 0)\ 1)\ (\mathsf{S} * (\mathsf{B}\ \mathit{fac}\ (\mathsf{C} - 1)))$

Applications group to the left, as in MORC. The factorial can be read from the inside out: $\mathsf{C} - 1$ is $\lambda n.\, n - 1$, $\mathsf{B}\ \mathit{fac}\ (\mathsf{C} - 1)$ is $\lambda n.\, \mathit{fac}\,(n-1)$, $\mathsf{S} *$ multiplies n by that, and $\mathsf{C}'\ \mathsf{IF}\ (\mathsf{C} = 0)\ 1$ builds the conditional around it. Notice that *fac* and *len* are still there, as names. They are not variables of any λ, so abstraction treats them as constants. The next section turns them into pointers.

8 Linking definitions

A program is a set of definitions that refer to each other, while the reducer needs a single graph. There are two ways to get from one to the other.

The first is to make the definitions into one big term. The program

$$\textbf{fn}\ f_1 = e_1;\ \ldots;\ \textbf{fn}\ f_n = e_n$$

becomes $(\lambda f_1 \ldots f_n.\, \mathit{main})\ e_1 \ldots e_n$, with a fixpoint combinator to let the e_i refer to themselves. This is how the early `.lam` programs were written, and it is expensive: bracket abstraction must then pass every f_i down through the whole program to each place that uses it, and most of the compiled code is Bs and Cs doing that routing.

The second, which SASL and Miranda used, is to keep the definitions apart. Each e_i is compiled on its own, leaving the names of definitions in it as constants. Then each definition is given a cell in the heap, and every occurrence of its name becomes a pointer to that cell. A recursive definition becomes a cycle in the graph, and mutual recursion is simply a larger cycle, so no fixpoint combinator is needed. `lam2graph.py` does this in four steps.

1. **Check.** Every definition is compiled, and any name left free in it that is not a definition is reported as unbound, whether or not the definition is used.
2. **Number.** Starting from *main*, definitions are numbered in the order in which they are first met: *main* is 0, the first definition it uses is 1, and so on through the code of each numbered definition in turn. Definitions that are never reached get no number.
3. **Replace.** Each name in the compiled code is replaced by a reference to the number of its definition.
4. **Write.** The reachable definitions are written to the graph file, in the format of section 9.

The second step means that a program pays only for what it uses. A library of definitions could be put in front of every program, and only the ones a program reaches would be loaded. That is how *cons* and *nil* are handled already.

Figure 4 shows the factorial after linking. Two details matter for the reducer.

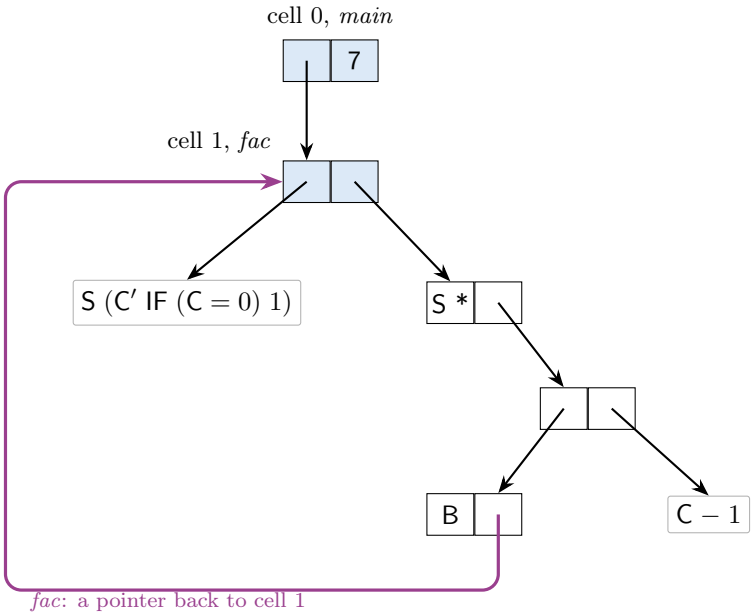


Figure 4: The factorial program after linking. Cell 1 holds the definition of *fac*, and the reference to *fac* inside it, the argument of B, is a pointer back to the same cell: recursion is a cycle. Cell 0 holds *main*, *fac* 7. Two subterms are drawn as boxes rather than as cells.

Cells, not indirections. The cell of a definition holds the definition's top application itself, not a pointer to it. A reference to *fac* therefore leads straight to an S application, and the reducer never has to step through an extra cell to find it. A definition whose code is not an application, such as `fn one = 1` or `fn main = f`, has no top cell to copy, so it is stored as I e, which costs one reduction the first time it is used.

Sharing comes for free. Every use of a definition points at the same cell, and the reducer overwrites a cell with its value when it reduces it. A definition without parameters, like *primes*, is therefore evaluated once, the first time it is needed, and every later use sees the part of it that has been computed so far. This is exactly the behaviour described in section 5, and it costs nothing extra to implement.

9 The graph file

The compiler hands the linked graph to the reducer as text, in postfix: each token pushes something on a stack, and the stack ends with the root of the graph. The tokens are:

token	effect
`I K S B C S' B* C' Y IF + - * / % = <`	push the combinator
`42`, `-42`	push the number
`@`	pop a and f; push a new cell (f, a), which is $f\ a$
`%`n	make n cells for the definitions; must come first
`$`k	push the cell of definition k
`=`k	pop a cell; copy its two words into definition k's cell
`#`	a comment, to the end of the line

At the end of the file exactly one word must be left on the stack, the root of the program. This is `fac.morc` compiled:

```
%2
```

```
# main
$1 7 @ =0
# fac
S C' IF @ C = @ 0 @ @ 1 @ @ S * @ B $1 @ C - @ 1 @ @ @ @ =1
# the program
$0
```

The first line makes two cells. `$1 7 @ =0` builds *fac* 7 and stores it in cell 0, *main*. The next line builds the code of *fac* and stores it in cell 1; inside it, `B $1 @` is B *fac* with a pointer to cell 1, which makes the cycle of fig. 4. Note that `$1` is used on the line of *main* before cell 1 has been filled. That is allowed, since a reference only needs the address of the cell, and the cell exists from the first line.

The loader. `graph.asm` reads this format with a loop of about 160 VM16 instructions, including the input routines it shares with the rest of the reducer. The heap is empty when loading starts, so the n cells made by `%`n are the first n cells of the heap, and cell k is at a fixed address, $HEAP + 4k$. `$`k is a single computation, and the loader needs no symbol table and no second pass. The numbers are chosen by the compiler, so names never reach the target machine.

Why text. A binary image of the heap would be smaller and faster to load. Text has three advantages: it can be read and written by hand, as in the tests of the loader; it does not depend on the reducer's memory layout; and it can be produced by a very simple program, including, one day, a compiler written in MORC itself (section 12).

10 Reduction

The reducer represents every value in one 16-bit word: a number n as the odd word $2n + 1$, a combinator as a small even code, and a cell as its even address in the heap. Cells are two words, the function and the argument of an application, with no tag. The reducer finds the leftmost combinator by following the function parts from the root, pushing each cell on a stack (the *spine*), and if the combinator has all its arguments, it rewrites the root of that application with the right-hand side of the combinator's rule. The details are in the companion tutorial [1]; here is what happens to MORC's factorial of 2, as `prototype.py --trace` prints it. Each line shows the graph after one reduction, and `@` marks the pointer back to the cell of *fac*.

```
 0      S (C' IF (C = 0) 1) (S * (B @ (C - 1))) 2
 1  S   C' IF (C = 0) 1 2 (S * (B (S (C' IF (C = 0) 1) @) (C - 1)) 2)
 2  C'  IF (C = 0 2) 1 (S * (B (S (C' IF (C = 0) 1) @) (C - 1)) 2)
 3  C   IF (= 2 0) 1 (S * (B (S (C' IF (C = 0) 1) @) (C - 1)) 2)
 4  =   IF (I 0) 1 (S * (B (S (C' IF (C = 0) 1) @) (C - 1)) 2)
 5  IF  I (S * (B (S (C' IF (C = 0) 1) @) (C - 1)) 2)
 6  S   I (* 2 (B (S (C' IF (C = 0) 1) (S * @)) (C - 1) 2))
 7  B   I (* 2 (S (C' IF (C = 0) 1) (S * (B @ (C - 1))) (C - 1 2)))
 ...
25  IF  I (* 2 (I (* (I 1) (I 1))))
26  *   I (* 2 (I (I 1)))
27  *   I (I 2)
```

Steps 1 to 5 are the test $n = 0$. S passes n to both halves of the definition, C′ and C route it into the comparison, the comparison yields 0 (written I 0, an indirection to the value), and IF chooses the **else** branch. Steps 6 and 7 build $2 * fac\,(2 - 1)$; at step 7 the B reaches the pointer to *fac* and applies the definition's own cell to the argument, which is the recursive call. The same pattern repeats for $n = 1$, and the multiplications finish the job. The argument $2 - 1$ is built

					VM16 instructions	
program	source	cells	reductions	GCs	setup	run
fac 7	`fac.morc`	14	87	0	3,022	6,509
	`fac.lam`	14	87	0	2,725	6,509
fib 20	`fib.morc`	19	240,795	45	3,717	16,796,640
	`fib.lam`	19	240,795	45	3,420	16,796,731
100th prime	`primes.morc`	72	177,761	49	12,680	13,146,080
	`primes.lam`	65	154,682	43	11,288	10,692,216

Table 3: MORC and `.lam` versions of the same programs.

once, as the cell `C - 1 2`, and shared by the comparison and the recursive call: at step 12 it is reduced to 1, and both uses see the result.

11 Measurements

Table 3 compares the MORC versions of three programs with the `.lam` versions of the companion tutorial, which are written in the terser notation but compute the same thing in the same way. The cells are the application cells of the compiled code, not counting the cell of each definition, and the VM16 instructions are counted by the C interpreter, split into loading the graph (*setup*) and everything after (*run*).

The same code. For *fac* and *fib*, MORC produces exactly the combinators of the `.lam` version and the same number of reductions. The only difference is that *main* is a definition of its own, which costs one cell and a few hundred instructions of loading. The runs of *fib* differ by 91 instructions out of 16.8 million.

The cost of !=. `primes.morc` tests divisibility with `x % h != 0`, which becomes

$$-\ 1\ (=\ (\%\ x\ h)\ 0),$$

while `primes.lam` uses the remainder directly as the condition. The extra comparison and subtraction, and the combinators that route their arguments, cost 15 % more reductions and 23 % more instructions. Writing `if x % h then ...` in MORC would give the same code as the `.lam` version, since any number other than 0 counts as true; the cleaner test is the one most programmers would write, and a compiler that simplified $-\ 1\ (=\ e\ 0)$ in a condition (section 12) could get the best of both.

Absolute speed. At about 70 VM16 instructions per reduction, and about 290 Z80 clock cycles per VM16 instruction, a 4 MHz Z80 performs some 200 reductions per second. Factorial of 7 takes about 0.7 seconds there, and the 100th prime would take about 16 minutes.

12 What MORC lacks

MORC is deliberately small, and several additions would make it more pleasant without changing the reducer.

Pattern matching on lists. Taking a list apart by applying it is correct but hard to read. A construct such as

```
fn length l = case l of nil -> 0 | h : t -> 1 + length t;
```

would translate directly into the application `l (\h t -> 1 + length t) 0`.

Local definitions. **let** ... **in** or **where** would allow helper functions and shared intermediate values inside a definition. A non-recursive **let** $x = a$ **in** b is $(\lambda x.\, b)\ a$; a recursive one needs either a fixpoint combinator or lambda lifting [3], which turns local functions into global definitions with extra parameters.

Characters, strings and output. At present a program can only produce a number. Turner's languages printed a lazy list of characters as it was computed, which allows a program to produce output as it runs, and to read its input the same way, as a lazy list that is filled in as it is consumed. This needs two new combinators in the reducer, one to print an element of the result list and one to read the next character, and character literals in the language.

Simplification. The compiler does no optimisation beyond Turner's rules. The conditional `if true then a else b` becomes `IF 1` a b, not a, and a negated comparison used as a condition is computed in full. A few rules applied to the core term before abstraction would remove both.

Types. MORC is untyped, so `1 2` and `(\x -> x) + 1` are only caught at run time. Hindley–Milner type inference, as in Miranda and Haskell, would catch them at compile time, at the cost of a much larger compiler.

A compiler in MORC. The compiler is about 650 lines of Python. With character input and output, it could be written in MORC itself, compiled once by the Python version, and then run on the reducer. Compiling the compiler with itself and comparing the result with the Python version's output would be a strong test of both, and it would make the system self-contained on the target machine, memory permitting: the reducer's heap holds 4,608 cells, and a compiler would need a large part of them.

13 Summary

MORC is a lazy functional language with a conventional surface, named definitions, infix operators and **if**–**then**–**else**, and almost no machinery behind it. Its grammar is a standard precedence grammar with application binding tightest. Its translation removes the surface syntax in one table, removes variables by bracket abstraction with Turner's combinators, and links definitions by pointers so that recursion becomes a cycle in the graph. The result is handed to a graph reducer of 2.4 kilobytes as postfix text. Each step is short, and only one of them, linking, needed anything new on the target machine: three tokens in the reducer's loader.

A Reference card

Programs	
`fn f x y = e;`	a definition; the program's value is *main*
`# ...`	a comment
Expressions, loosest first	
`\x y -> e`	a function; the body extends to the right
`if c then a else b`	0 is false, any other number true
`a == b`, `=`, `!=`, `<`, `<=`, `>`, `>=`	comparisons, 1 or 0; they do not chain
`h : t`	a list, grouped to the right
`a + b`, `a - b`	grouped to the left; `-a` is negation
`a * b`, `a / b`, `a % b`	`/` and `%` truncate towards zero
`f x y`	application, `(f x) y`; binds tightest
`(e)`, `(+)`, `(:)`	brackets; an operator as a function
`42`, **true**, **false**, **nil**	15-bit numbers; 1; 0; the empty list
Lists	
`l (\h t -> a) b`	a with head h and tail t if l is not empty, else b
Commands	
`./run prog.morc`	compile and run
`python3 lam2graph.py -t prog.morc`	show the compiled combinators
`python3 lam2graph.py prog.morc`	write `prog.graph`
`python3 prototype.py --trace prog.morc`	the graph after every reduction

References

[1] Combinator graph reduction: a tutorial introduction. `graph/graph-tutorial.pdf` in this repository.

[2] S. Marlow, editor. *Haskell 2010 Language Report*, 2010.

[3] S. L. Peyton Jones. *The Implementation of Functional Programming Languages*. Prentice Hall, 1987.

[4] D. A. Turner. A new implementation technique for applicative languages. *Software—Practice and Experience*, 9(1):31–49, 1979.

[5] D. A. Turner. Miranda: a non-strict functional language with polymorphic types. In *Functional Programming Languages and Computer Architecture*, LNCS 201, pages 1–16. Springer, 1985.